

VIBEMEGOOD

Generator Implementation Spec

A technical audit and developer implementation guide for the VibeMeGood lineup generator — covering all identified bugs, missing features, and architecture upgrades.

Prepared for	Scope	Date	Priority
VibeMeGood Developer	lineup-factory.ts + 4 supporting files	June 11, 2026	11 findings 1 Critical · 2 High

OVERVIEW

What This Document Is

This spec is a complete implementation guide based on a full code audit of the VibeMeGood lineup generator. It covers every file involved in lineup construction — the route handler, projection engine, entry simulator, payout tables, and pick'em math library. Each section identifies the problem, explains why it matters, and gives the developer exact instructions to fix it.

Do not skip the Critical or High severity items. They affect every user on every slate right now. The Medium and Low items are real improvements but can be sequenced after the urgent fixes ship.

FINDINGS SUMMARY

11 Issues Across 6 Files

Area	Issue	Severity
Payout Tables	6-pick Power = 40× (should be 37.5×); all Flex tiers wrong in both files	Critical
Correlation Split	Generator uses naive 1.10×/1.03× multipliers; real Cholesky model is never called by generator	High
Greedy Fill	No combination search; props scored individually, not as lineup combinations	High
Portfolio Monte Carlo	Shared-prop correlation across lineups ignored — independence assumed	Medium
Stacks Feature Gap	PrizePicks Stacks (live July 2025) not supported — no config flag or logic	Medium
Composite Score Units	max_ev and balanced_growth mix dollar-denominated EV with percentage-denominated edge	Medium
Projection Sourcing	Entirely self-sourced — no external projection signal to cross-validate edge	Medium
Same-Player Corr.	1.10× boost is stat-type-agnostic and has no calibration basis	Medium
Exposure Tracking	Cap has no effect on early lineups — only kicks in after threshold is crossed	Low

Area	Issue	Severity
p99 for Poisson Stats	Normal formula ($\text{mean} + 2.33\sigma$) applied to Poisson-distributed stats	Low
Goblin/Demon Mults.	Fixed 1.08×/0.92× multipliers with no calibration basis	Low

FIX 1 OF 8 · CRITICAL

Payout Tables — Fix Both Files

⚠ Every EV number shown to users is wrong right now.

PrizePicks changed 6-pick Power Play from 40× to 37.5× and restructured all Flex payouts. Both payout files in this codebase still reflect the old schedule. Users see overstated expected value on every 5- and 6-pick entry.

Files to Change

File	Change Required
artifacts/api-server/src/lib/payout/tables.ts	Update POWER_PAYOUTS and FLEX_PAYOUTS
lib/analytics/src/pickem-math.ts	Update POWER_PAYOUTS, FLEX_PAYOUTS, and ENTRY_TYPES break-even values

Correct Power Play Multipliers

Replace the existing POWER_PAYOUTS object in both files:

```
export const POWER_PAYOUTS = {
  2: 3,
  3: 6,
  4: 10,
  5: 20,
  6: 37.5, // was 40 — PrizePicks changed this
};
```

Correct Flex Multipliers

Replace the existing FLEX_PAYOUTS object in both files:

```
export const FLEX_PAYOUTS = {
  3: { 3: 3, 2: 1.0 },
  4: { 4: 6, 3: 1.5 },
  5: { 5: 10, 4: 2, 3: 0.4 },
  6: { 6: 25, 5: 2, 4: 0.4 },
};
```

Update Break-Even Values in pickem-math.ts

The ENTRY_TYPES object in pickem-math.ts has hardcoded break-even values. After updating the payout tables, recalculate these using the flexBreakEven() bisection function already in the file — it reads from FLEX_PAYOUTS so it will self-correct once the table is updated. For Power Play break-evens, the formula is $(1/\text{multiplier})^{(1/\text{picks})}$. Updated values:

```
"6-pick-power": { multiplier: 37.5, breakEven: Math.pow(1/37.5, 1/6) }, // ≈ 0.5416
"5-pick-flex": { multiplier: ..., breakEven: flexBreakEven(5) }, // recalculated
"6-pick-flex": { multiplier: ..., breakEven: flexBreakEven(6) }, // recalculated
```

Verification

- After updating, confirm: a 6-pick Power at \$10 stake shows $EV = 0.375 \times 37.5 - 10 = \4.0625 for 6 legs at 60% hit rate (not \$4.40)
- Confirm Flex 6-pick shows 25× for all-hit, 2× for 5/6, 0.4× for 4/6

FIX 2 OF 8 · HIGH

Wire the Real Correlation Model Into the Generator

Two correlation systems exist — only one is correct, and it's never used by the generator.

entry-simulator.ts contains a fully working Cholesky-based Monte Carlo correlation model with sport/position-aware pair correlations (NFL QB+WR=0.65, NBA same-team=0.40, etc.). The lineup generator in lineup-factory.ts uses a completely separate naive system: same player pair = 1.10×, same game = 1.03×. These two systems do not share any code or data.

Files to Change

File	Change Required
artifacts/api-server/src/routes/lineup-factory.ts	Replace calcCorrelationFactor() with call to simulateEntry()
artifacts/api-server/src/lib/simulation/entry-simulator.ts	No changes needed — this is the correct model
artifacts/api-server/src/lib/simulation/correlation.ts	No changes needed — correct pair correlations already defined

How the Real Model Works

- simulateEntry({ legs, runs, multiplier }) takes an array of SimLeg objects
- Each SimLeg needs: playerName, statType, line, side, modelProb, sport, team, gameld, mean, stdDev
- It builds a correlation matrix from sport/team/position/gameld context
- Cholesky-decomposes the matrix, samples correlated normals, runs N Monte Carlo iterations
- Returns trueJointProbability — use this instead of the naive product of individual probabilities

What to Replace

In lineup-factory.ts, find calcPowerEV() and calcCorrelationFactor(). Remove calcCorrelationFactor() entirely. In the lineup construction loop (step 6), after picks[] is finalized for a lineup, call simulateEntry() to get the true joint probability:

```
import { simulateEntry } from '../lib/simulation/entry-simulator';

// After picks[] is filled for a lineup:
const simResult = simulateEntry({
  legs: picks.map(p => ({
    playerName: p.playerName,
```

```

statType: p.statType,
line: p.ppLine,
side: p.direction === 'more' ? 'over' : 'under',
modelProb: p.hitProbability,
sport: p.sport,
team: String(p.teamId ?? ''),
gameId: String(p.gameId ?? ''),
mean: p.projMean ?? p.ppLine, // add projMean to ScoredProp
stdDev: p.projStdDev ?? 1.0, // add projStdDev to ScoredProp
})),
runs: 5000, // 5k is fast enough per lineup; 10k for final display
multiplier: POWER_MULT[picks.length] ?? 10,
});
const pHit = simResult.trueJointProbability;

```

Add projMean and projStdDev to ScoredProp

The ScoredProp object needs two new fields to pass to simulateEntry. These values already exist in the database query result (row.proj.projectedValue and row.proj.stdDev). Add them when building allScoredProps:

```

projMean: pMean, // already computed as row.proj?.projectedValue
projStdDev: pStd, // already computed as row.proj?.stdDev

```

Performance Note

Running simulateEntry at 5,000 iterations per lineup for 25 lineups = 125,000 iterations total. This takes roughly 20–50ms in Node.js and is acceptable. Do not run it for every candidate during greedy fill — only run it once after picks[] is finalized per lineup.

FIX 3 OF 8 · HIGH

Fix Composite Score Unit-Mixing

max_ev and balanced_growth mix dollars and percentages in a single formula.

max_ev: $ev + edge * 0.05$ — at a \$10 stake, EV is ~\$0-\$5 and $edge * 0.05$ for a 20% edge is 1.0. These are not the same unit. Normalizing both to a 0–100 scale fixes the rankings.

File to Change

File	Change Required
<code>artifacts/api-server/src/routes/lineup-factory.ts</code>	Replace <code>calcCompositeScore()</code> switch cases for <code>max_ev</code> and <code>balanced_growth</code>

Corrected Formula — max_ev

Normalize EV to a 0–100 scale using the stake as the divisor:

```
case 'max_ev':  
  // evScore: EV as % of stake (0–100 scale)  
  const evScore = (ev / stake) * 100;  
  const edgeScore2 = Math.max(0, edge ?? 0); // edge already in percentage points  
  return evScore * 0.70 + edgeScore2 * 0.30;
```

Corrected Formula — balanced_growth

```
case 'balanced_growth':  
  const evNorm = (ev / stake) * 100; // normalize to same scale as prob * 100  
  return evNorm * 0.50 + prob * 100 * 0.50;
```

FIX 4 OF 8 · MEDIUM

Portfolio Monte Carlo — Add Shared-Pick Correlation

The portfolio simulator treats all lineups as independent even when they share picks.

If Player A appears in 5 of your 10 lineups, those 5 lineups are correlated — they all win or lose together based on Player A's outcome. The current simulator draws each lineup independently, which understates bust risk and overstates diversification.

File to Change

File	Change Required
artifacts/api-server/src/routes/lineup-factory.ts	Replace monteCarloPortfolio() with correlated version

Replacement Function

```
function monteCarloPortfolio(lineups, totalStake, iterations = 10000) {
  // Build pick-hit map: for each iteration, decide once per unique ppLineId
  const allPickIds = [...new Set(lineups.flatMap(lu => lu.picks.map(p => p.ppLineId)))]
  let breakEven = 0, profitable = 0
  for (let i = 0; i < iterations; i++) {
    // One hit/miss draw per unique prop – shared across all lineups
    const hitMap = new Map(allPickIds.map(id => {
      const prop = allScoredProps.find(p => p.ppLineId === id)
      return [id, Math.random() < (prop?.hitProbability ?? 0.5)]
    }))
    let payout = 0
    for (const lu of lineups) {
      const allHit = lu.picks.every(p => hitMap.get(p.ppLineId))
      if (allHit) payout += lu.grossPayout
    }
    if (payout >= totalStake) breakEven++
    if (payout > totalStake) profitable++
  }
  return { probBreakEven: breakEven/iterations, probProfitable: profitable/iterations }
}
```

Note: allScoredProps needs to be in scope — pass it as a parameter or use closure.

FIX 5 OF 8 · MEDIUM

Add PrizePicks Stacks Support

PrizePicks launched Stacks in July 2025. VibeMeGood has no support for them.

Stacks allow the same player to appear up to 3 times in one lineup using different stat types (e.g., LeBron Points + LeBron Rebounds + LeBron Assists). Currently the generator query hard-codes `pickCategory = 'player'` and has no same-player deduplication exceptions for stacks.

Files to Change

File	Change Required
<code>artifacts/api-server/src/routes/lineup-factory.ts</code>	Add <code>stacksAllowed</code> config flag + relax same-player deduplication
Zod config schema (top of <code>lineup-factory.ts</code>)	Add <code>stacksAllowed: z.boolean().default(false)</code>
<code>artifacts/api-server/src/lib/simulation/correlation.ts</code>	Add same-player, different-stat pair correlations

Config Schema Change

```
// Add to configSchema:
stacksAllowed: z.boolean().default(false),
maxStacksPerPlayer: z.number().int().min(1).max(3).default(2),
```

Deduplication Logic Change

Currently the greedy fill loop checks: `if (picks.some(p => p.playerId === candidate.playerId)) continue;` — this blocks all same-player picks. Change it to:

```
// Count how many times this player already appears in current lineup
const playerCountInLineup = picks.filter(p => p.playerId === candidate.playerId).length;
const maxAllowed = cfg.stacksAllowed ? cfg.maxStacksPerPlayer : 1;
if (playerCountInLineup >= maxAllowed) continue;

// Also block exact same stat duplication (same player, same stat)
if (picks.some(p => p.playerId === candidate.playerId &&
  p.statType === candidate.statType)) continue;
```

Correlation — Same Player, Different Stats

Add to `pairCorrelation()` in `correlation.ts`. Same player, different stats should use stat-aware correlations:

```
// Same player, different stat – add BEFORE the sport checks
if (a.playerId && a.playerId === b.playerId) {
  const pts = s => s.toLowerCase().includes('points');
  const rebs = s => s.toLowerCase().includes('rebound');
  const ast = s => s.toLowerCase().includes('assist');
  if ((pts(aStat) && rebs(bStat)) || (rebs(aStat) && pts(bStat))) return 0.45;
  if ((pts(aStat) && ast(bStat)) || (ast(aStat) && pts(bStat))) return 0.50;
  if ((rebs(aStat) && ast(bStat)) || (ast(aStat) && rebs(bStat))) return 0.40;
  return 0.35; // same player, unknown stat combo – mildly positive
}
```

Note: `LegContext` in `entry-simulator.ts` does not currently carry `playerId`. Add `playerId` to the `LegContext` interface and pass it through from `ScoredProp`.

FIX 6 OF 8 · LOW

Exposure Tracking — Fix Early-Lineup Blindspot

The exposure cap has no effect on the first several lineups in every session.

The check is: $(\text{playerCounts}[\text{id}] + 1) / \text{cfg.numEntries} > \text{maxExposure}$. On lineup 1, `playerCounts` is empty, so this check always passes for the first several lineups until enough lineups are built to push the count over the threshold. A player with 40% max exposure in a 25-lineup set can appear in lineups 1–10 freely before the cap kicks in.

File to Change

File	Change Required
<code>artifacts/api-server/src/routes/lineup-factory.ts</code>	Change exposure cap denominator from <code>numEntries</code> to <code>luldx + 1</code>

Change

Find the exposure check in the greedy fill loop and update the denominator:

```
// Current (wrong):  
const totalFuture = cfg.numEntries;  
  
// Fix – use lineups built so far + 1 (rolling denominator):  
const totalSoFar = lineups.length + 1; // +1 for the lineup being built  
const afterPlayer = (playerCounts[candidate.playerId] ?? 0) + 1;  
if (afterPlayer / totalSoFar > cfg.maxPlayerExposure + 0.001) continue;
```

This makes the exposure cap enforce a rolling constraint rather than a forward-looking one. A player at 40% cap cannot exceed 40% of lineups generated so far at any point during construction.

FIX 7 OF 8 · LOW

p99 Ceiling Fix for Poisson Stats

The ceiling estimate uses a normal formula on stats that follow a Poisson distribution.

$p99 = \text{mean} + 2.33 \times \text{stdDev}$ works for normally distributed stats (points, yards, assists). For sparse counting stats (home runs, goals, strikeouts, turnovers), the distribution is Poisson. At low lambda values, the Poisson 99th percentile diverges significantly from a normal approximation.

File to Change

File	Change Required
artifacts/api-server/src/lib/projection/compute.ts	Replace p99 formula for Poisson-family stats

Change

```
// Current (line ~325 in compute.ts):
const p99 = sourceLabel === 'prior_only'
? null
: Math.round((mean + 2.33 * effectiveStd) * 100) / 100;

// Fix – use Poisson quantile for sparse counting stats:
function poissonP99(lambda: number): number {
  // Walk CDF until cumulative >= 0.99
  let cum = 0, k = 0;
  const expLambda = Math.exp(-lambda);
  let term = expLambda;
  while (cum < 0.99 && k < 200) {
    cum += term;
    k++;
    term *= lambda / k;
  }
  return k;
}

const p99 = sourceLabel === 'prior_only' ? null
: distFamily === 'poisson'
? poissonP99(mean)
: Math.round((mean + 2.33 * effectiveStd) * 100) / 100;
```

FIX 8 OF 8 · MEDIUM

Projection Sourcing — Add External Signal Blend

The projection engine is entirely self-sourced — it finds edge only vs. its own internal model.

There is no external projection signal to cross-validate. If the internal model is wrong about a player, there is nothing to correct it. The fix is to blend an external signal (Vegas-implied team total or FantasyPros consensus) into the mean before committing to a projection.

Recommended Approach — Vegas-Implied Scaling

The gamesTable already stores game total and spread (synced from The Odds API). The impliedTotalFactor() function in factors.ts already converts game total + spread to an implied team total and scales the projection. This is already in the factor stack. The improvement is to make this factor apply more aggressively for high-confidence implied totals:

```
// In factors.ts — impliedTotalFactor:  
// Current: caps the factor at ±12% (0.88-1.12)  
// Increase cap to ±18% when the implied total deviates strongly from baseline:  
const MAX_FACTOR = impliedTotal && Math.abs(impliedTotal - baseline) > 5 ? 1.18 : 1.12;  
const MIN_FACTOR = impliedTotal && Math.abs(impliedTotal - baseline) > 5 ? 0.82 : 0.88;
```

Recommended Approach — FantasyPros Blend (Future)

For a stronger external signal, add a fantasypros_projections table to the DB schema and sync FantasyPros consensus projections alongside the existing sync jobs. Then blend in computeProjection():

```
// After computing weighted mean from game logs:  
if (fpProjection) {  
  // 60% internal model, 40% FantasyPros consensus  
  mean = mean * 0.60 + fpProjection * 0.40;  
}
```

This is optional scope — implement the Vegas factor improvement first since the data infrastructure already exists.

ARCHITECTURE

What the Codebase Gets Right

These components are well-built. Do not refactor them.

Projection engine (compute.ts): decay-weighted mean, Bayesian shrinkage, calibration, full factor stack — solid.

Entry simulator Cholesky model (entry-simulator.ts + correlation.ts): correct math, well-structured.

Bulk query architecture in lineup-factory.ts: no N+1 queries, clean batching.

Bias adjustment: user's personal hit-rate vs model delta is a genuinely differentiated feature.

GPP mode composite score: $(\text{ceiling} / \text{ownership}) \times \text{edge}$ — clean, dimensionless, well-designed.

Config schema: flexible and covers the right user-facing controls.

Combo stat reconstruction: Pts+Rebs computed from components per game date, only complete dates.

DELIVERY

Implementation Sequence

Ship in this order. The Critical fix unlocks correct EV for all users immediately. The two High fixes are the biggest performance improvements. Medium items after.

Phase 1 — Ship Now	Critical
[] Update POWER_PAYOUTS in tables.ts: set 6 → 37.5	
[] Update FLEX_PAYOUTS in tables.ts: replace all tiers with current PrizePicks schedule	
[] Mirror both changes in pickem-math.ts	
[] Recalculate ENTRY_TYPES break-even values using flexBreakEven() — already in the file	
[] Deploy and verify: 6-pick Power EV at \$10 stake with 60% legs = \$4.0625 (not \$4.40)	
Phase 2 — Sprint 1	High
[] Add projMean and projStdDev fields to ScoredProp type	
[] In lineup construction loop, call simulateEntry() after picks[] is finalized	
[] Use simResult.trueJointProbability as the lineup pHit	
[] Remove calcCorrelationFactor() — it is fully replaced	
[] Add playerId to LegContext in correlation.ts	
[] Normalize max_ev and balanced_growth composite score formulas	
[] Replace monteCarloPortfolio() with shared-pick correlated version	
Phase 3 — Sprint 2	Medium
[] Add stacksAllowed and maxStacksPerPlayer to config schema	
[] Relax same-player deduplication in greedy fill to allow stacks	
[] Add same-player, different-stat correlations to pairCorrelation()	
[] Fix exposure tracking denominator (rolling, not forward-looking)	
[] Fix p99 for Poisson stats using poissonP99() function	

[] Increase Vegas-implied factor cap in factors.ts

Questions on any spec item? The audit covers the full reasoning behind each finding. If a code path has changed since this audit, verify the file structure before applying the fix.